Практическая работа: «Интеграция React-клиента с Node.js API (Emoji Finder)»

Цель работы: Научиться организовывать взаимодействие между React-приложением (на TypeScript) и бэкенд-сервисом. Вы отработаете полный цикл: от описания типов данных до обработки состояний загрузки, ошибок и отображения контента.

Исходные данные

У вас уже есть верстка на **React + TypeScript**. Бэкенд (Node.js + Express) с API /api/emojis уже написан и готов к работе. Сервер умеет отдавать список эмодзи (максимум 20 штук) и фильтровать их по параметру q (поиск по названию и ключевым словам).

Задача

Вам необходимо внедрить в существующий проект функционал поиска и отображения эмодзи, соблюдая следующие требования:

1. **Типизация:** описать интерфейсы TypeScript для данных, которые приходят с сервера. Никаких any.
2. **Разделение ответственности:** Логика запроса к API должна находиться в отдельном файле (сервис/клиент), а не внутри компонента.
3. **Обработка состояний:** Компонент должен уметь показывать:
    - Состояние загрузки (loading).
    - Обработку ошибок (например, если сервер выключен).
    - Рендер списка данных.
4. **Интерактивность:** реализовать поиск, который отправляет запрос на сервер при каждом изменении текста в поле ввода.
5. **Стилизация вывода:** красиво отобразить карточку каждого эмодзи (сам символ, название, ключевые слова).

Пошаговое руководство

Шаг 1. Проверка окружения и запуск сервера

Прежде чем писать код на React, убедитесь, что бэкенд работает.

1. Откройте терминал в папке проекта бэкенда.
2. Запустите файл **start.bat** двойным кликом или из консоли.
3. Дождитесь сообщения: Сервер запущен на **http://localhost:3000**.
4. Проверьте в браузере: откройте **http://localhost:3000/api/emojis**. Вы должны увидеть JSON-массив.

Если видите ошибку — проверьте, установлен ли Node.js и нет ли опечаток в путях. Батник сам установит зависимости, если их нет.

Шаг 2. Подготовка зависимостей

Если в вашем React-проекте еще нет библиотеки для запросов, установите её:

npm install axios

(Можно использовать нативный fetch, но axios удобнее для типизации и перехвата ошибок).

Шаг 3. Создание сервиса API (Обязательный этап)

Создайте файл src/api/emojiApi.ts. Это научит вас не засорять компоненты логикой запросов.

```ts
// src/api/emojiApi.ts
import axios from 'axios';

// 1. Описываем типы данных (строго типизируем ответ)
export interface IEmojiItem {
  id: number;
  emoji: string;       // Символ эмодзи
  title: string;
  keywords: string;
}

const API_URL = 'http://localhost:3000/api/emojis';

/**
 * Функция получения эмодзи.
 * @param query - поисковый запрос (может быть пустым)
 */
export const getEmojis = async (query?: string): Promise<IEmojiItem[]> => {
  try {
    // Формируем параметры запроса
    const params = query ? { q: query } : undefined;

    // Отправляем GET-запрос
    const response = await axios.get<IEmojiItem[]>(API_URL, { params });

    return response.data;
  } catch (error) {
    console.error('Ошибка при запросе к API:', error);
    throw error; // Пробрасываем ошибку выше, чтобы компонент мог её обработать
  }
};
```

Критерии проверки этого шага:

- Интерфейсы описаны явно, без any.
- Функция асинхронная, возвращает Promise.
- Логика обработки ошибок вынесена сюда (или проброшена дальше).

Шаг 4. Реализация компонента

Создайте или обновите компонент src/components/EmojiFinder.tsx. Внедрите его в свою верстку там, где должен быть поиск.

```tsx
import React, { useState, useEffect } from 'react';
import { getEmojis, type IEmojiItem } from './api/emojiApi';

const App: React.FC = () => {
  const [emojis, setEmojis] = useState<IEmojiItem[]>([]);
  const [searchTerm, setSearchTerm] = useState('');
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState<string | null>(null);

  // Эффект, который срабатывает при изменении searchTerm
  useEffect(() => {
    fetchData();
  }, [searchTerm]);

  const fetchData = async () => {
    setLoading(true);
    setError(null);
    try {
      const data = await getEmojis(searchTerm);
      setEmojis(data);
    } catch (err) {
      // Здесь мы ловим ошибку, которую пробросил сервис
      setError('Не удалось загрузить данные. Проверьте, запущен ли сервер (start.bat).');
    } finally {
      setLoading(false);
    }
  };

  // Рендер состояний


  if (error) {
    return (
      <div style={{ padding: '20px', color: 'red', fontWeight: 'bold', textAlign: 'center' }}>
        {error}
      </div>
    );
  }
```

```
  return (
    <div style={{ maxWidth: '800px', margin: '0 auto', padding: '20px' }}>
      <h1>Поиск эмодзи</h1>

      {/* Поле поиска */}
      <div style={{ marginBottom: '30px' }}>
        <input
          type="text"
          placeholder="Введите название или ключевое слово..."
          value={searchTerm}
          onChange={(e) => setSearchTerm(e.target.value)}
          style={{ padding: '10px', width: '100%', boxSizing: 'border-box' }}
        />
        {searchTerm && (
          <button
            onClick={() => setSearchTerm('')}
            style={{ marginTop: '10px', cursor: 'pointer' }}
          >
            Очистить поиск
          </button>
        )}
      </div>

      {loading ? <div style={{ padding: '20px', textAlign: 'center' }}>Загрузка эмодзи...</div> : null}
      {/* Сетка карточек */}
      <div style={{ display: 'grid', gridTemplateColumns: 'repeat(auto-fill, minmax(120px, 1fr))', gap: '15px' }}>
        {emojis.length === 0 ? (
          <p>Эмодзи не найдены</p>
        ) : (
          emojis.map((emoji) => (
            <div
              key={emoji.id}
              style={{
                border: '1px solid #ddd',
                padding: '15px',
                borderRadius: '8px',
                textAlign: 'center',
                backgroundColor: '#f9f9f9',
                transition: 'transform 0.2s'
              }}
              onMouseEnter={(e) => (e.currentTarget.style.transform = 'scale(1.05)')}
              onMouseLeave={(e) => (e.currentTarget.style.transform = 'scale(1)')}
            >
              {/* Вывод самого эмодзи */}
              <div style={{ fontSize: '40px', lineHeight: '1' }}>{emoji.emoji}</div>

              <h3 style={{ margin: '10px 0 5px', fontSize: '16px' }}>{emoji.title}</h3>
              <small style={{ color: '#666', display: 'block' }}>
                {emoji.keywords}
              </small>
            </div>
          ))
        )}
      </div>
    </div>
  );
};

export default App;
```

Критерии проверки этого шага:

- Компонент использует useState и useEffect.

- Нет "магических строк" внутри JSX (стили вынесены в объекты или инлайн-объекты для простоты).
- При вводе текста в поле поиска запрос уходит на сервер (можно проверить во вкладке Network в браузере).
- При отсутствии данных показывается сообщение "Эмодзи не найдены".

Чек-лист для сдачи работы

- Сервер запущен через start.bat.
- В React-проекте нет ошибок TypeScript (интерфейсы описаны верно).
- При вводе слова «fire» в поиске на странице остаются только соответствующие эмодзи (проверка работы бэкенда).
- Видны состояния «Загрузка...» и обработка ошибки (попробуйте остановить сервер и обновить страницу).
- Карточки эмодзи имеют базовую стилизацию и реагируют на наведение (опционально).
- Код сервиса (emojiApi.ts) отделен от кода компонента.

Возможные проблемы и решения

Проблема	Решение
Ошибка «Network Error»	Проверьте, запущен ли сервер (start.bat) и слушает ли он порт 3000.
TypeScript ругается на типы	Проверьте, что интерфейсы совпадают с тем, что реально возвращает сервер.